
Waiting for everyone to join
TU257
Information Systems
Marisa Llorens

Information Systems Review

Week 6

TU257

Data Models

- Specify concepts that are used to describe the database. These can be categorised as:
  - ***High Level (Conceptual)***
  - ***Representational (Logical)***
  - ***Low Level (Physical)***

Conceptual model

- Identify
  - Entities
  - Relationships
  - Attributes
- Create an ER diagram where the structure of database can be easily understood
- Document all requirements and assumptions

ER Diagrams Process Flow

- Requirements
- Rough ER
  - Entities
  - Relationships
  - Basic attributes
- Key based ER diagram
  - Remove many to many relationships
  - Identify all PKs
  - Identify FKs
- Fully Attributed ER diagram

Along the way identify assumptions and write them down

Design tips

- Meaningful Tables
  - Each row contains one instance of entity or relationship
  - What's in this table?
- Separate column for independently accessed data
  - Name, Surname, Address
- Each cell contains only one piece of data
  - Phone number
  - Add extra if exact number or separate into own table

Design tips

- Each table has a key
  - One PK
  - Could have more candidate keys
  - PK cannot be NULL
- Tables are related using foreign keys (FK)
  - Referential integrity constraint

Design tips

- Minimize empty cells
  - Columns with many NULL values should be placed in separate table
  - This new table will only contain entries for non null values
  - Optional relationships

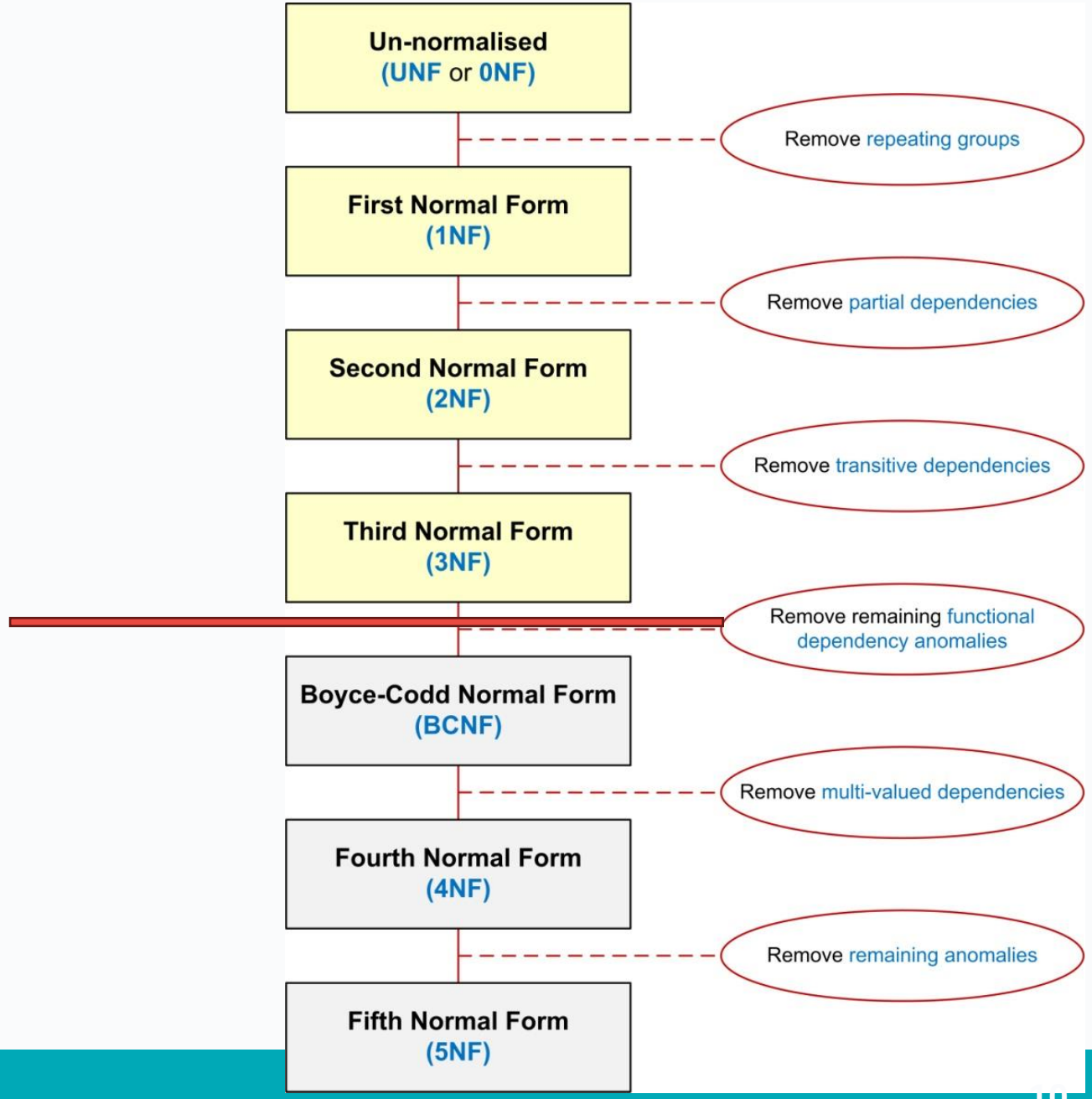
Design tips

- Many to many relationships
    - Remove and create bridge entity

- Document requirements and assumptions

Logical Model

- Relational model
- Normalization process
  - Reduce redundancy
  - Avoid data anomalies
  - Avoid logical inconsistencies
  - Increase efficiency



How it all fits together

“Use an entity relation diagram (ERD) to provide the big picture, or macro view, of an organization’s data requirements and operations. This is created through an iterative process that involves identifying relevant entities, their attributes and their relationships.”

“Normalization procedure focuses on characteristics of specific entities and represents the micro view of entities within the ERD.”

Review exercises Lab 4

A university library needs a new dbms. The system needs to handle users borrowing and returning books and keep track of the overall status of the library. The library is located across several sites and all students can borrow up to 10 books at a time from any of the sites. For some popular books, the library keeps multiple copies across different sites to facilitate students' access. Details of site manager, authors and publishers are also kept.

Extra requirements:

- Each site has one manager and managers can only manage one site.
- Assume books only have one author (for now).

Review exercises Lab 4
Entities

Initial List

- Books
- Authors
- Publishers
- Library_Sites
- Managers
- Users
- Library

Did we cover everything?

Should everything stay on the list?

How do we solve the issue with more than one copy of the same book?

Review exercises Lab 4
Entities

Initial List

- Books
- Authors
- Publishers
- Library_Sites
- Managers
- Users
- ~~Library~~
- Book_objects

Did we cover everything?

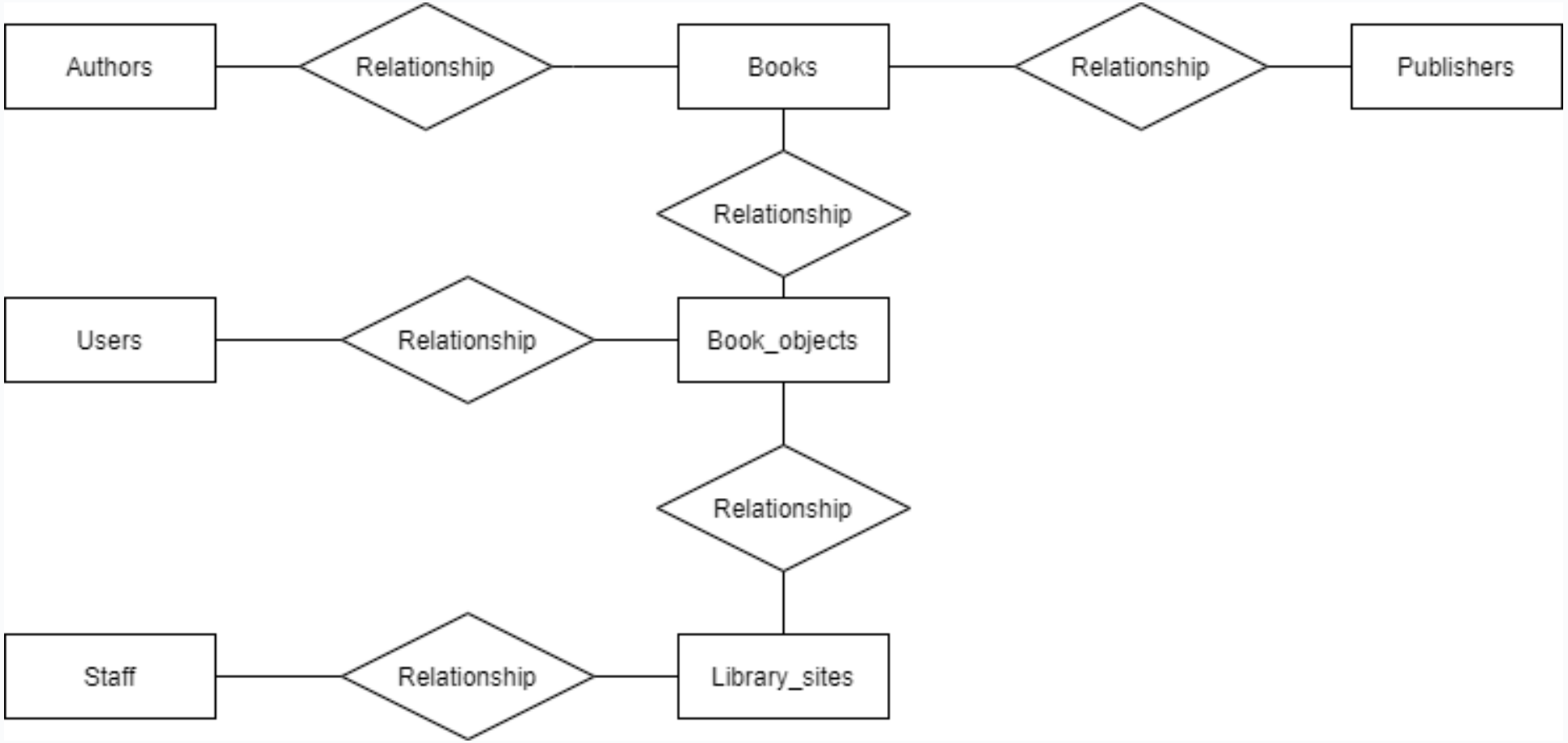
Should everything stay on the list?

How do we solve the issue with more than one copy of the same book?

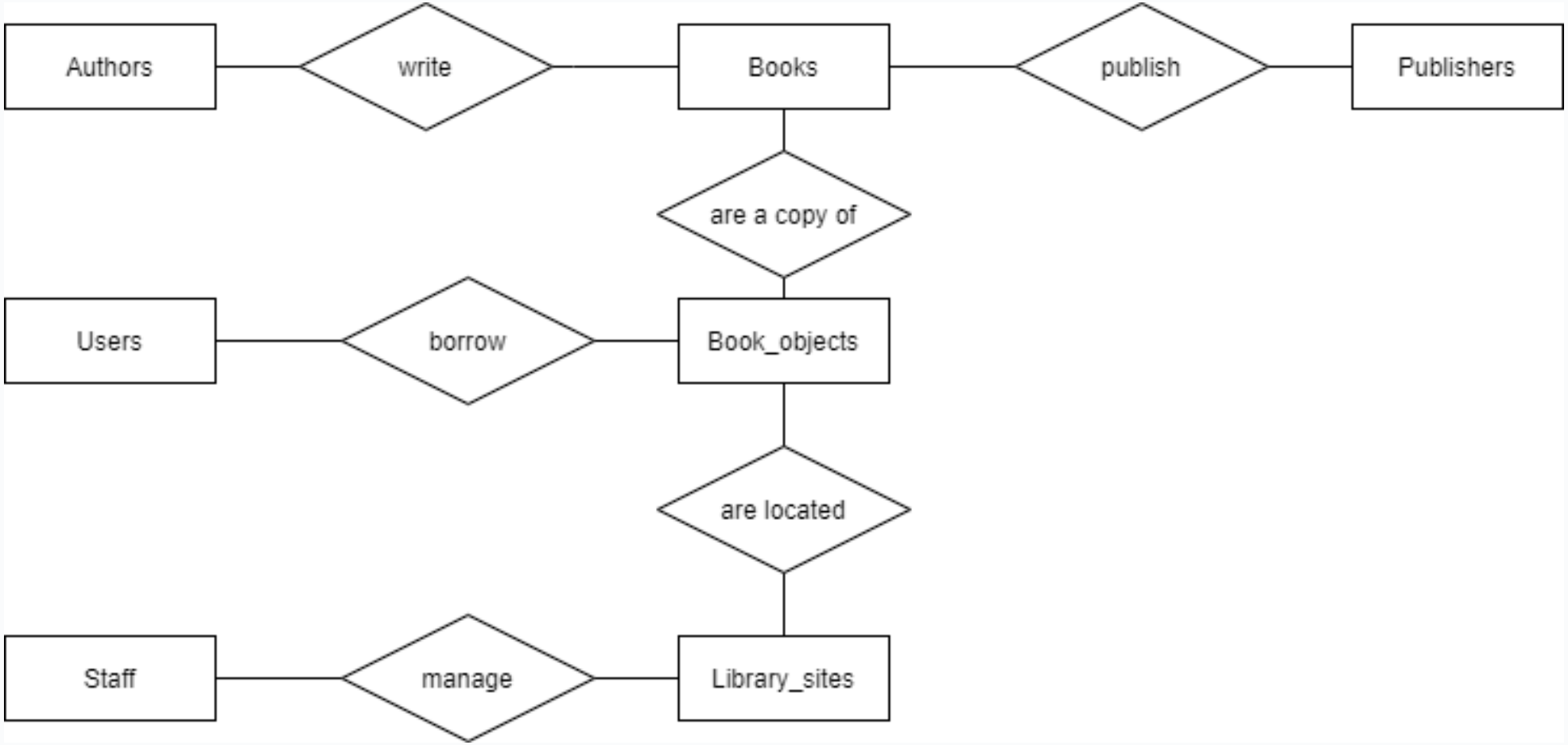
Review exercises Lab 4
Find relationships

	Books	Authors	Publishers	Library_Sites	Managers	Users	Book_Objects
Books	X						
Authors	write	X					
Publishers	publish		X				
Library_Sites				X	Are managed		store
Managers				manage	X		
Users						X	borrow
Book_objects	Are copies of			Are stored		Get borrowed	X

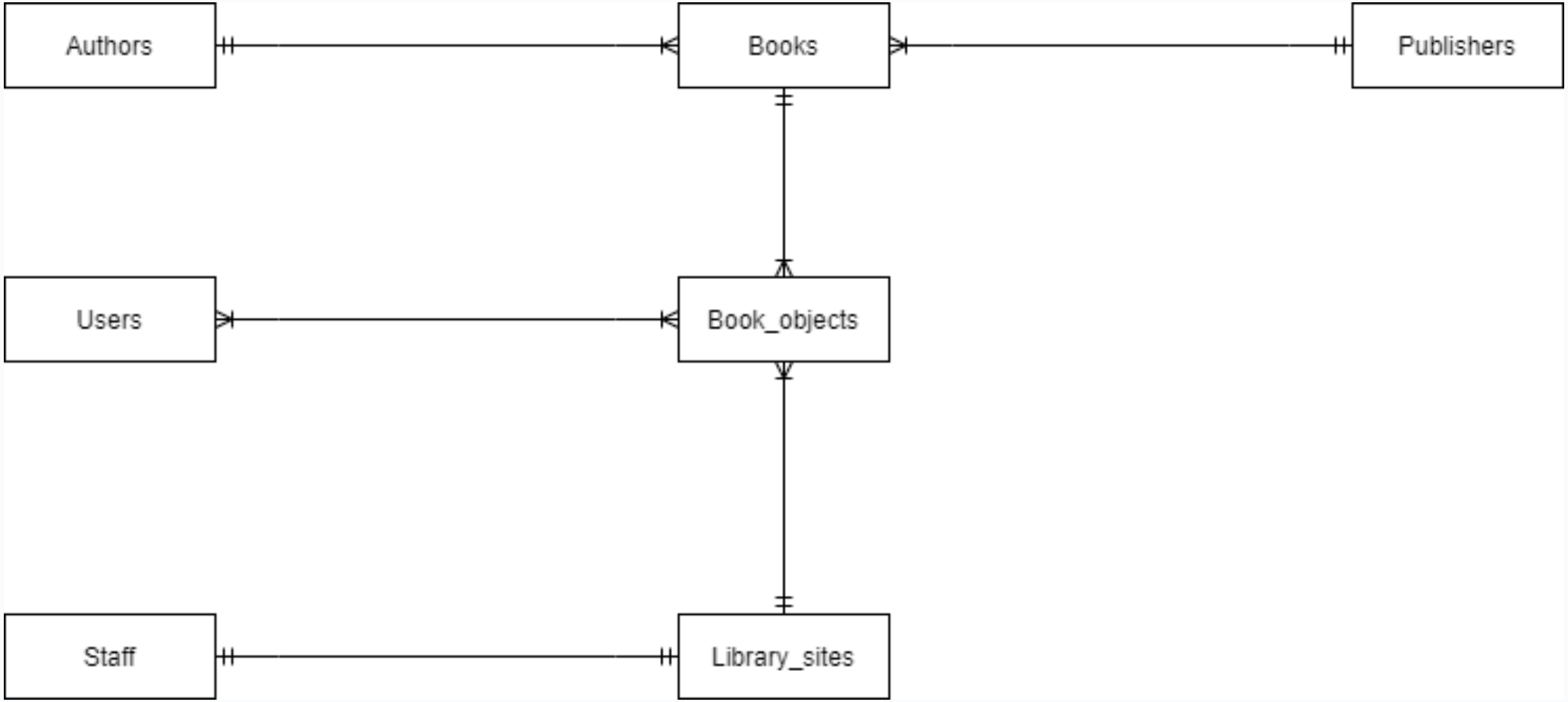
Review exercises Lab 4
Rough ER Diagram



Review exercises Lab 4
Rough ER Diagram



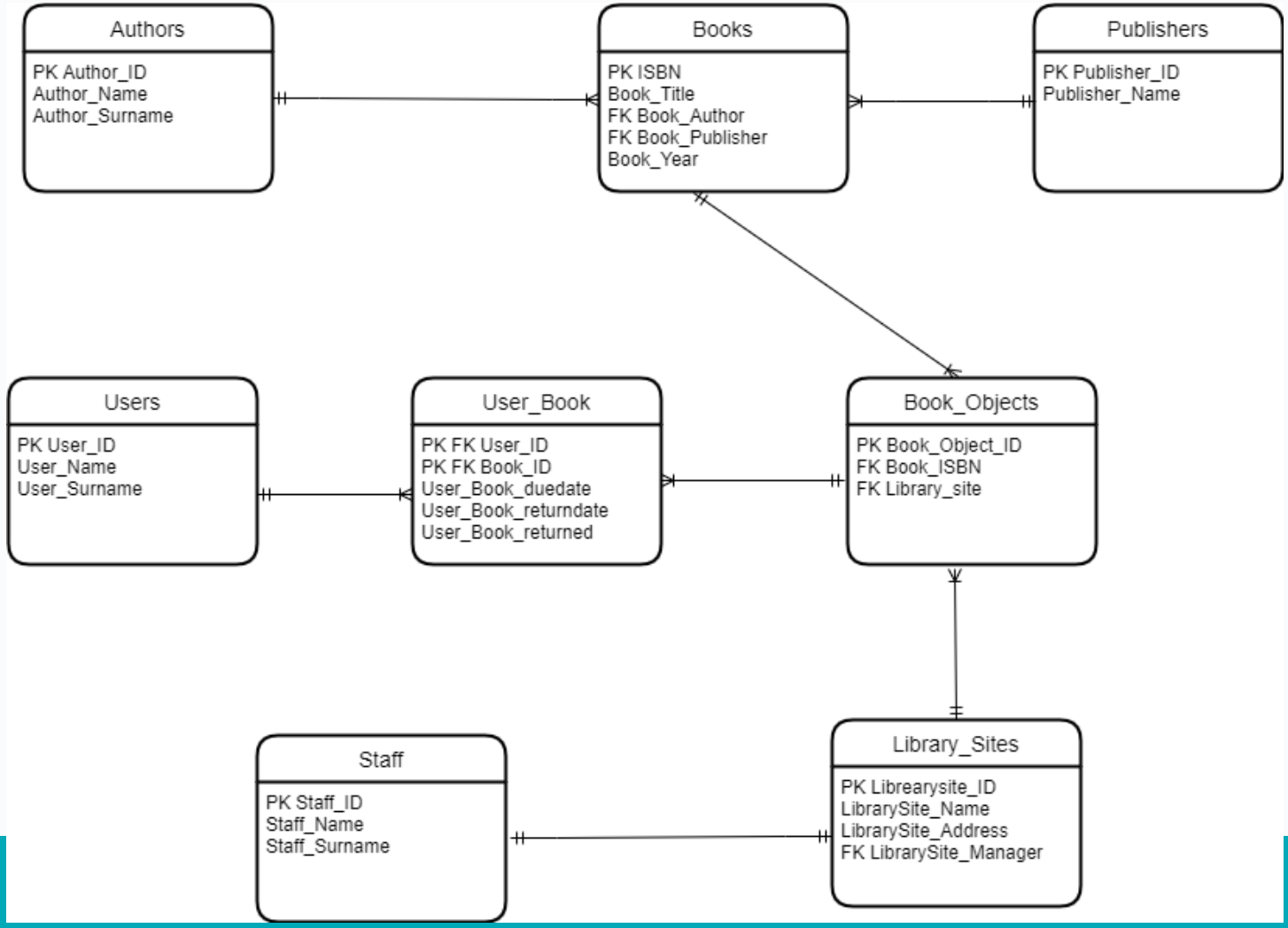
Review exercises Lab 4 Cardinality



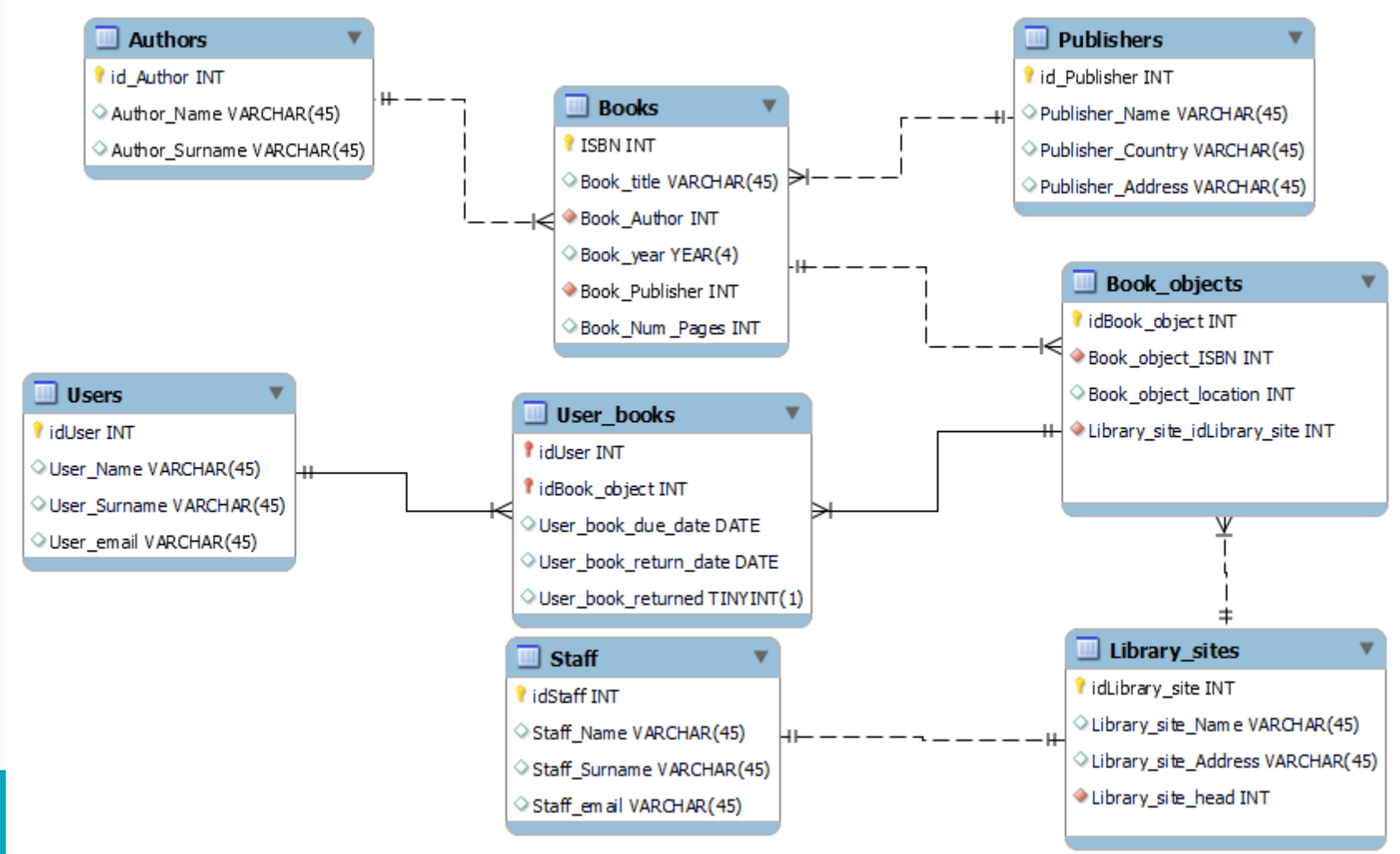
Review exercises Lab 4
Primary keys

- Author_ID
- Book_ISBN
- Publisher_ID
- User_ID
- BookObject_ID
- LibrarySite_ID
- Staff_ID

Review exercises Lab 4
Key based ER diagram



Review exercises Lab 4
Fully Attributed ER diagram



Review exercises Lab 4 Questions

1. What is the aim of this system?

   Do we need information about staff?

2. What about books written by more than one author?

3. What if we want to know if a book is available?

Review exercises Lab 4 Discussion

1. Design should match a specific purpose, do we need the managers included?

2. In order to include more than one author per book, we have a many to many relationship with an extra bridge entity

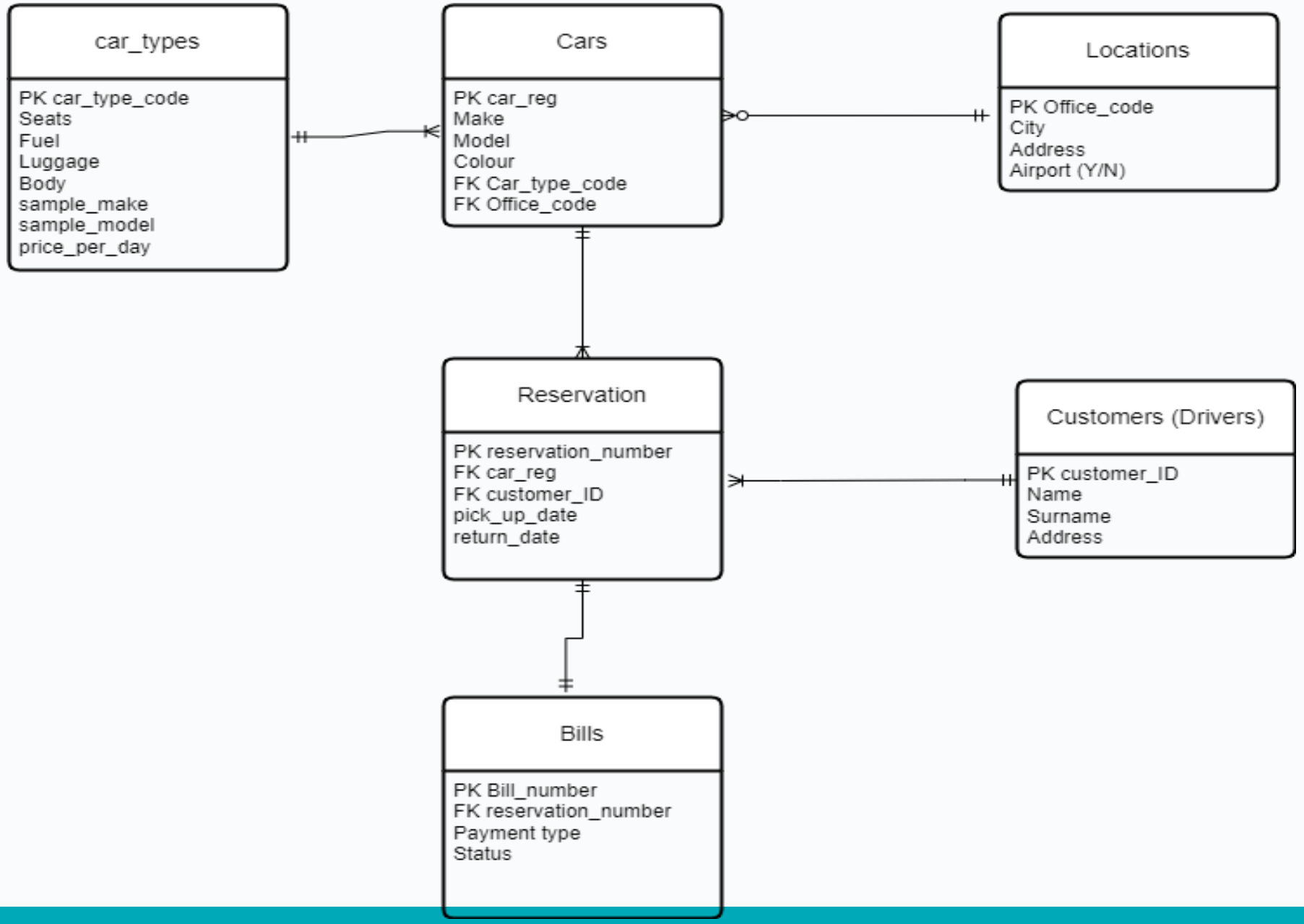
3. To check if a book is available we are not going to add anything to the schema, we will use code to calculate how many of the copies are available.

Extra issues
Different editions of the same book have different ISBN.

Review exercises Assumptions

- A car rental company needs a new DBMS. The company needs to deal with customers' details, reservations and returns. A customer can book several cars but each reservation is made to only one customer. A fleet of cars is stored at each location and cars have to be picked up and returned to the same location.

Sample Solution 1



Assumptions 10 minutes breakout room

1.

2.

3.

4.

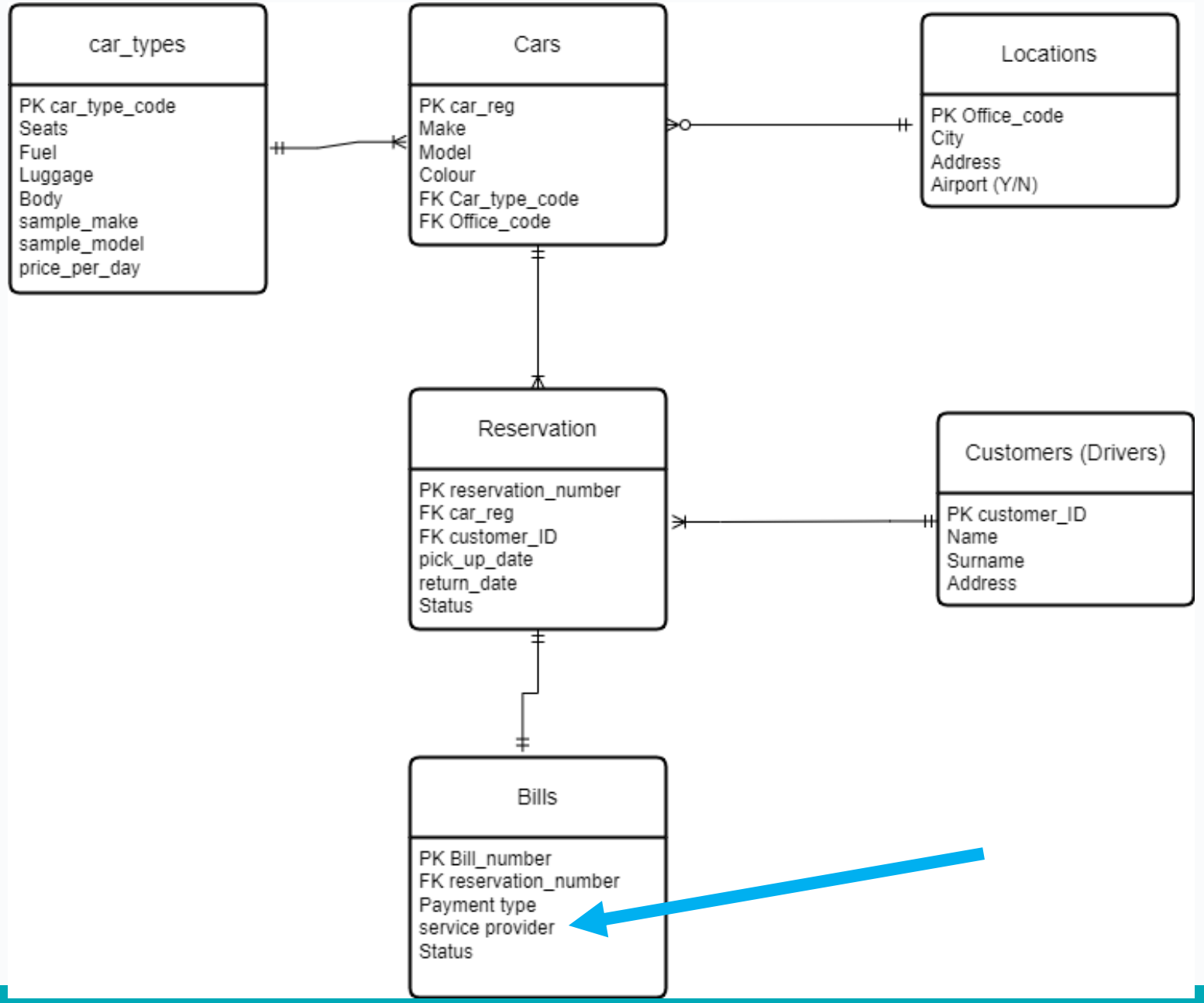
5.

6.

Assumptions

1. Car characteristics are determined by their car type. For each car type there may be a number of cars available.
2. Car types have fixed prices. Sample make and model are fixed per car type.
3. Each location can store a number of cars but cars can only be allocated to one location.
4. Location and customer address are given using the Eircode.
5. Each reservation only contains data about one car and one customer (no extra drivers). Each reservation has a corresponding bill (one) for the full amount.
6. Each bill contains details about one reservation only.
7. Each customer can book different cars on different reservations.
8. Each car can be booked on different reservations.

Sample Solution 2



Assumptions

1. Car characteristics are determined by the car type.
2. Car types have fixed prices. Sample make and model are fixed per car type.
3. Each location can store a number of cars but cars can only be allocated to one location.
4. Location and customer address are given using the Eircode.
5. Each reservation only contains data about one car and one customer (no extra drivers). Each reservation has a corresponding bill (one) for the full amount.
6. Each bill contains details about one reservation only.
7. **Each payment type has an specified service provider.**

Normalisation

CARS(**car_reg**, make, model, colour, car_type_code, office_code)

CAR_TYPES(**car_type_code**, seats, fuel, luggage, body, sample_make, sample_model, price_per_day)

LOCATIONS(**office_code**, city, address, airport)

RESERVATIONS(**reservation_number**, car_reg, customer_ID, pick_up_date, return_date, status)

BILLS(**bill_number**, reservation_number, payment_type, service_provider, status)

CUSTOMERS(**customer_ID**, name, surname, address)

Normalisation 1 NF

No repeating groups. Each relation checked.

CARS1(**car_reg**, make, model, colour, car_type_code, office_code)

CAR_TYPES1(**car_type_code**, seats, fuel, luggage, body, sample_make, sample_model, price_per_day)

LOCATIONS1(**office_code**, city, address, airport)

RESERVATIONS1(**reservation_number**, car_reg, customer_ID, pick_up_date, return_date, status)

BILLS1(**bill_number**, reservation_number, payment_type, service_provider, status)

CUSTOMERS1(**customer_ID**, name, surname, address)

Normalisation 2 NF

No partial dependencies as no composite key used. Each relation checked.

CARS2(**car_reg**, make, model, colour, car_type_code, office_code)

CAR_TYPES2(**car_type_code**, seats, fuel, luggage, body, sample_make, sample_model, price_per_day)

LOCATIONS2(**office_code**, city, address, airport)

RESERVATIONS2(**reservation_number**, car_reg, customer_ID, pick_up_date, return_date, status)

BILLS2(**bill_number**, reservation_number, payment_type, service_provider, status)

CUSTOMERS2(**customer_ID**, name, surname, address)

Normalisation 3 NF

No transitive dependencies in the following relations:

CARS3(**car_reg**, make, model, colour, car_type_code, office_code)

CAR_TYPES3(**car_type_code**, seats, fuel, luggage, body, sample_make, sample_model, price_per_day)

LOCATIONS3(**office_code**, city, address, airport)

RESERVATIONS3(**reservation_number**, car_reg, customer_ID, pick_up_date, return_date, status)

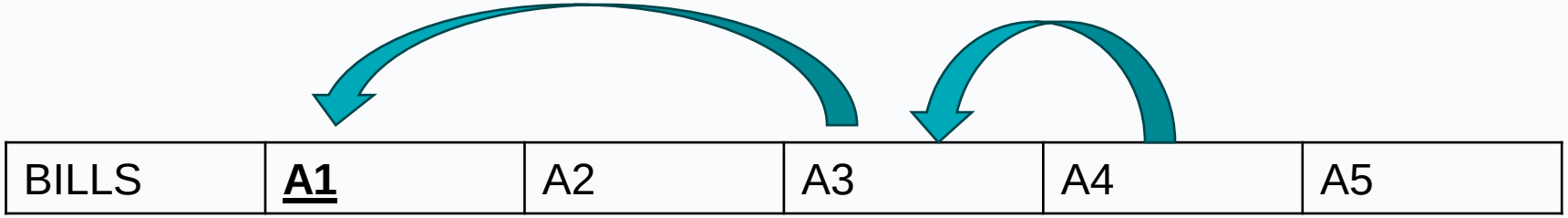
CUSTOMERS3(**customer_ID**, name, surname, address)

Normalisation 3 NF

Transitive dependency found between service provider and payment_type.

From assumptions: Each payment type has an specified service provider.

BILLS2(**bill_number**, reservation_number, payment_type, service_provider, status)



Normalisation 3 NF

Transitive dependency found between service provider and payment_type.

From assumptions: Each payment type has an specified service provider.

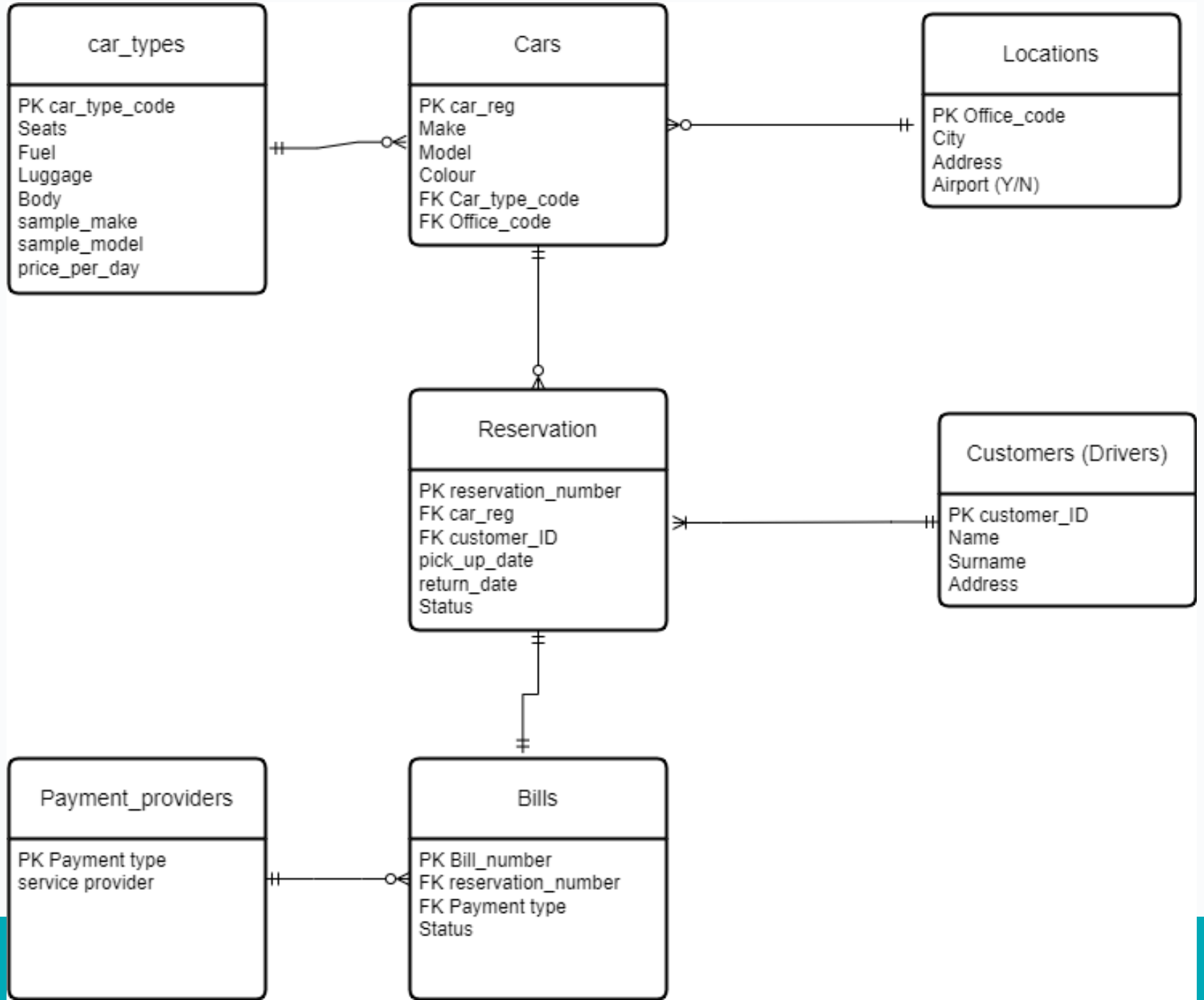
BILLS2(**bill_number**, reservation_number, payment_type, status)

PAYMENT_PROVIDER(payment_type, service_provider)

Sample Solution 2 after normalisation

Optionality also reviewed

List of assumptions should be updated



Normalisation examples

Lab 5 exercise 1

Exercise 1

A car rental company has hired you to design a relational schema to hold their booking's data. Customers provide their details as well as the type of car they want to rent and any extra services they would like to include in the booking (for example GPS or child seat). A sample of their data is shown on Table 1. Normalise the system to the third normal form.

BookingID	Car Type	base rent	custID	Name	DOB	Age	Cust Address	Date of booking	Service name	service charge
1	Sedan	50	101	John	1990-05-15	33	123 Main St, City	2023-10-17	GPS, Child Seat	10,15
2	SUV	80	102	Sarah	1985-07-21	38	456 Elm St, Town	2023-10-16	Extra Driver	15
3	Hatchback	40	103	Michael	1995-03-10	28	789 Oak St, Village	2023-10-15	Wash & Vacuum	20
4	Convertible	70	104	Emily	1988-11-30	34	101 Pine Rd, County	2023-10-14	GPS, Wash & Vacuum	10,20
5	Truck	90	105	Daniel	1980-09-05	43	222 Cedar Ln, City	2023-10-13	GPS	10
6	Sedan	50	106	Jessica	1992-04-20	31	333 Birch Ave, Town	2023-10-12	Child Seat	15
7	Hatchback	40	107	Brian	1987-08-12	36	444 Oak Rd, City	2023-10-11	Wash & Vacuum	20
8	SUV	80	108	Amanda	1993-01-25	30	555 Elm St, Village	2023-10-10	Child Seat	15
9	Convertible	70	109	Robert	1997-06-03	26	666 Maple Ln, County	2023-10-09	GPS, Extra Driver	10,15
10	Sedan	50	110	Olivia	1996-12-18	26	777 Pine St, Town	2023-10-08	Wash & Vacuum	20

Table 1 Car Rental Data

Lab 5 exercise 1

Exercise 1

A car rental company has hired you to design a relational schema to hold their booking's data. Customers provide their details as well as the type of car they want to rent and any extra services they would like to include in the booking (for example GPS or child seat). A sample of their data is shown on Table 1. Normalise the system to the third normal form.

BookingID	Car Type	base rent	custID	Name	DOB	Age	Cust Address	Date of booking	Service name	service charge
1	Sedan	50	101	John	1990-05-15	33	123 Main St, City	2023-10-17	GPS, Child Seat	10,15
2	SUV	80	102	Sarah	1985-07-21	38	456 Elm St, Town	2023-10-16	Extra Driver	15
3	Hatchback	40	103	Michael	1995-03-10	28	789 Oak St, Village	2023-10-15	Wash & Vacuum	20
4	Convertible	70	104	Emily	1988-11-30	34	101 Pine Rd, County	2023-10-14	GPS, Wash & Vacuum	10,20
							222 Cedar			

CARRENTAL(BookingID, car_type, base_rent, cust_ID, Name, DOB, Age, Cust_address, date_booking, service, service_charge)

Lab 5 exercise 1

- Unnormalised as more than one element in services.

CARRENTAL(**BookingID**, car_type, base_rent, cust_ID, Name, DOB, Cust_address, date_booking, (service, service_charge)*)

Lab 5 exercise 1

- To convert to 1NF (remove repeating groups)

CARRENTAL1(**BookingID**, car_type, base_rent, cust_ID, Name, DOB, Cust_address, date_booking)

SERVICES1 (**BookingID, service**, service_charge)

Lab 5 exercise 1

- To convert to 2NF (remove partial dependencies)
- No composite key so already in 2NF

CARRENTAL2(**BookingID**, car_type, base_rent, cust_ID, Name, DOB, Cust_address, date_booking)

Lab 5 exercise 1

- To convert to 2NF (remove partial dependencies)
- Partial dependency found (service_charge depends on service but not on bookingID)

SERVICEBOOKING2 (**BookingID, service**)

SERVICEDETAILS2(**service**, service_charge)

Lab 5 exercise 1

- To convert to 3NF (remove transitive dependencies)
- Transitive dependencies found

CARRENTAL2(**BookingID**, car_type, base_rent, cust_ID, Name, DOB, Cust_address, date_booking)

Lab 5 exercise 1

- To convert to 3NF (remove transitive dependencies)
- Transitive dependencies found

CARRENTAL3(**BookingID**, car_type, cust_ID, date_booking)

CARDETAILS(**car_type**, base_rent,)

CUSTOMER3(**cust_ID**, Name, DOB, Cust_address)

Lab 5 exercise 1

- To convert to 3NF (remove transitive dependencies)
- No transitive dependencies found

SERVICEBOOKING3 (**BookingID, service**)

SERVICEDETAILS3(**service**, service_charge)

Lab 5 exercise 1

CARRENTAL3(**BookingID**, car_type, cust_ID, date_booking)

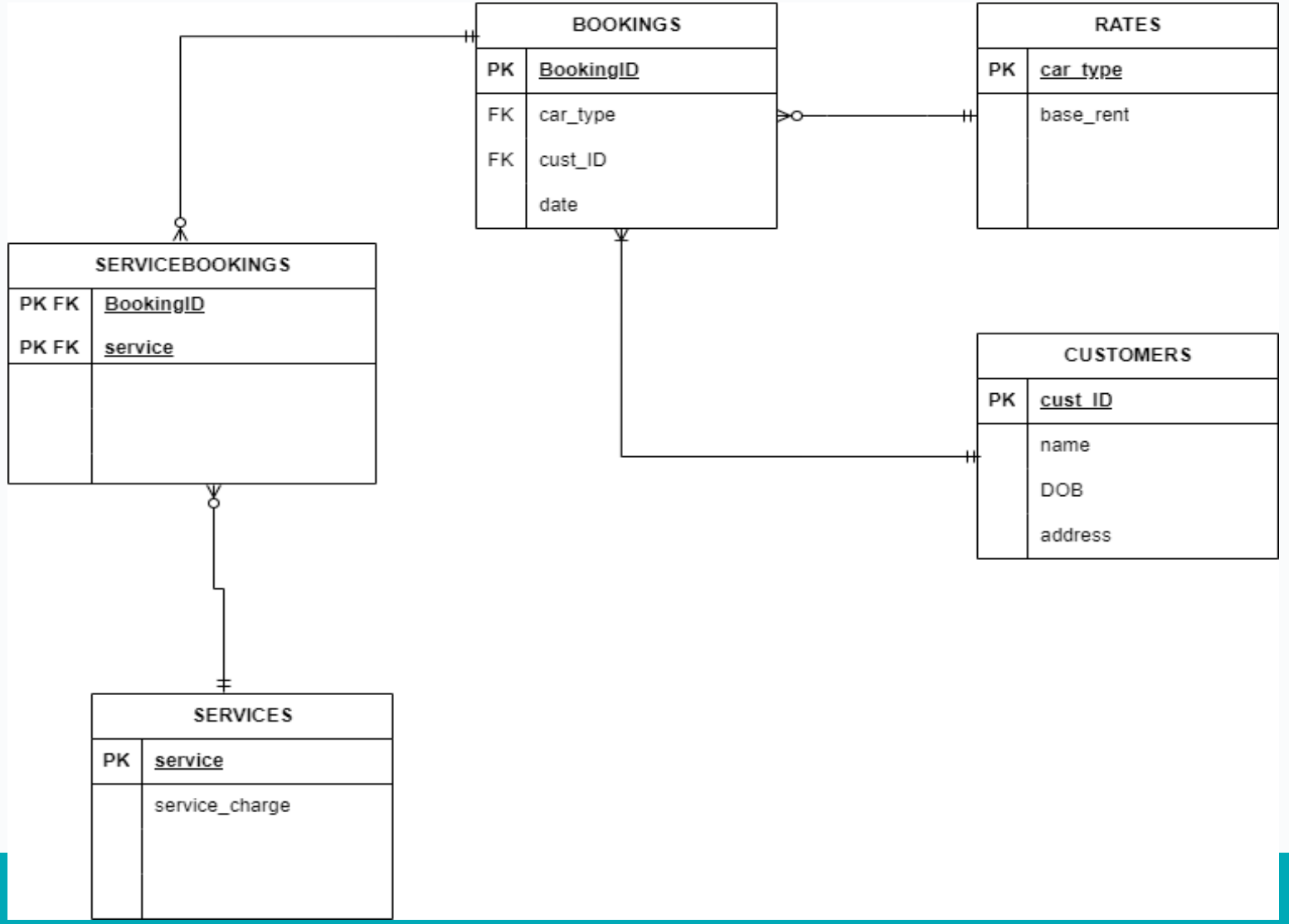
CARDETAILS(**car_type**, base_rent,)

CUSTOMER3(**cust_ID**, Name, DOB, Cust_address)

SERVICEBOOKING3 (**BookingID, service**)

SERVICEDETAILS3(**service**, service_charge)

Lab 5 exercise 1



Exercise 2 LAB 5

Consider a company keeping track of projects, employees and departments.
WORK(proj_Name, proj_manager, empl_id, hours, empl_name, budget, startDate, salary, empl_manager, empl_dept, rating)
Convert our table into a db 3NF.

projName	projManager	Empl_id	hours	Employee name	budget	Start_date	salary	Empl_manager	Empl_department	rating
Gamma	Smith	101	25	Jones	2000	01/01/2021	2000	O'Reilly	10	9
Gamma	Smith	121	32	Lee	2000	01/01/2021	2500	Kennedy	12	
Gamma	Smith	104	40	Kelly	2000	01/01/2021	1000	O'Reilly	10	6
Delta	Murphy	101	25	Jones	5000	02/10/2021	2000	O'Reilly	10	6
Gamma	Smith	133	32	Lee	2000	01/01/2021	2500	Kennedy	12	
Delta	Murphy	111	15	Bowles	5000	02/10/2021	1500	Kennedy	12	7

Exercise

Requirements

- Each project has a unique name.
- Employee and manager names are not unique
- Many employees can work in each project and each employee can be assigned to more than one project.
- Hours refers to the number of hours an employee is assigned to a particular project.
- Budget and start_date refer to the total budget for the project and project start date.
- Salary refers to the employee’s salary

Exercise

Requirements

- …
- Empl_manager refers to the name of the employee's manager, which may be different to the project's manager.
- Empl_department refers to the employee's department. Department names are unique and the employee's manager is the employee's department manager.
- Rating refers to the employee's performance in a specific project.

Solution Exercise 2 LAB 5

WORK(proj_Name, proj_manager, empl_id, hours, empl_name, budget, startDate, salary, empl_manager, empl_dept, rating)

1NF

WORK(project_name, project_manager, empl_id, hours, empl_name, budget, startDate, salary, empl_manager, empl_dept, rating)

- Identify Remove repeating groups

WORK((project_name, project_manager, budget, startDate)*, hours, (empl_id, empl_name, salary, empl_manager, empl_dept)*, rating)

1NF

WORK((project_name, project_manager, budget, startDate)*, hours, (empl_id, empl_name, salary, empl_manager, empl_dept)*, rating)

- Remove repeating groups
- Choose PK

1NF

PROJECT1(**project_name**, project_manager, budget, startDate)

WORK1(**project_name**, **empl_id**, hours, rating)

EMPLOYEE1(**empl_id,** emply_name, salary, empl_manager, empl_dept)

2NF

Identify Partial dependencies

PROJECT1(**project_name**, project_manager, budget, startDate)

WORK1(**project_name**, **empl_id**, hours, rating)

EMPLOYEE1(**empl_id,** emply_name, salary, empl_manager, empl_dept)

2NF

No composite Key=>No Partial dependencies

PROJECT1(**project_name**, project_manager, budget, startDate)

EMPLOYEE1(**empl_id,** emply_name, salary, empl_manager, empl_dept)

2NF

No Partial dependencies

Hours and rating depend on both the project_name and the empl_id

WORK1(**project_name**, **empl_id**, hours, rating)

2NF

PROJECT2(**project_name**, project_manager, budget, start_date)

EMPLOYEE2(**empl_id**,empl_name, salary, empl_manager, empl_department)

WORK2(**project_name, empl_id**, hours, rating)

3NF

- Identify Transitive dependencies

PROJECT2(**project_name**, project_manager, budget, start_date)

EMPLOYEE2(**empl_id**, empl_name, salary, empl_manager, empl_department)

WORK2(**project_name, empl_id**, hours, rating)

3NF

- Transitive dependencies

empl_department-> empl_manager

EMPLOYEE2(**empl_id**,empl_name, salary, empl_manager, empl_department)

3NF

- Transitive dependencies empl_department-> empl_manager

EMPLOYEE3(**empl_id**, empl_name, salary, empl_department)

DEPARTMENT3(**empl_department**, empl_manager)

3NF

- Transitive dependencies empl_department-> empl_manager

EMPLOYEE3(**empl_id**, empl_name, salary, empl_department)

DEPARTMENT3(**empl_department**, empl_manager)

3NF

EMPLOYEE3(**empl_id**, empl_name, salary, empl_department)

DEPARTMENT3(**empl_department**, empl_manager)

PROJECT3(**project_name**, project_manager, budget, start_date)

WORK3(**project_name, empl_id**, hours, rating)

QUESTIONS?